



МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Новосибирский государственный технический университет»



**НГТУ
НЭТИ**

**Факультет прикладной
математики и информатики**

Кафедра теоретической и прикладной информатики
Практическое задание № 2
по дисциплине «УРАВНЕНИЯ МАТЕМАТИЧЕСКОЙ ФИЗИКИ»

РЕШЕНИЕ НЕЛИНЕЙНЫХ НАЧАЛЬНО-КРАЕВЫХ ЗАДАЧ

Вариант 8

ПМ-34 РУБЦОВ АРТЁМ

ПМ-34 КУДЮКОВ МАКСИМ

Преподаватели

ЗАДОРОВНИЙ АЛЕКСАНДР ГЕННАДЬЕВИЧ

ЛЕОНОВИЧ ДАРЬЯНА АЛЕКСАНДРОВНА

Новосибирск, 2026

Содержание	
1 Цель работы	3
2 Задание	3
3 Теоретическая часть.....	3
4 Тестирование	4
4.1 Равномерная сетка.....	4
4.1.1 $ux, t = x^2; \lambda u = 1; \sigma = 1; fx, t = -2; ux, 0 = x^2$	4
4.1.2 $ux, t = t^2; \lambda u = 1; \sigma = 1; fx, t = 2t; ux, 0 = 0$	4
4.1.3 $ux, t = x^2; \lambda u = u; \sigma = 1; fx, t = -6x^2; ux, 0 = x^2$	5
4.1.4 $ux, t = x^2 + t; \lambda u = u; \sigma = 1; fx, t = 1 - 6x^2 - 2t; ux, 0 = x^2$	5
4.2 Неравномерная сетка	5
4.2.1 $ux, t = x^2; \lambda u = 1; \sigma = 1; fx, t = -2; ux, 0 = x^2$	5
4.2.2 $ux, t = x^2; \lambda u = \cos u; \sigma = 1; fx, t = 4x^2 \sin x^2 - 2 \cos x^2; ux, 0 = x^2$	5
5 Выводы	6
6 Текст программы	6
6.3 Main.cpp	6
6.4 Basis.h	7
6.5 Basis.cpp	8
6.6 Mesh.h	8
6.7 Mesh.cpp	9
6.8 Func.h	10
6.9 Func.cpp	10
6.10 Matrix.h	13
6.11 MatrixOperations.h	13
6.12 MatrixOperations.cpp	13
6.13 LU.h	14
6.14 LU.cpp	15
6.15 Element.h	15
6.16 Element.cpp	16
6.17 SimpleIteration.h	20
6.18 SimpleIteration.cpp	21
6.19 Newton.h	27
6.20 Newton.cpp	28

1 Цель работы

Разработать программу решения нелинейной одномерной краевой задачи методом конечных элементов. Провести сравнение метода простой итерации и метода Ньютона для решения данной задачи.

2 Задание

5. Уравнение: $-\text{div}(\lambda(u) \text{grad } u) + \sigma \frac{\partial u}{\partial t} = f$. Базисные функции – линейные.

3 Теоретическая часть

Уравнение:

$$-\text{div}(\lambda(u) \text{grad}(u)) + \sigma \partial u / \partial t = f,$$

С краевыми и начальными условиями:

$$u(S_1) = u_g$$

$$u(t_0) = u_0$$

Производную по времени на каждом временном слое t_s будем аппроксимировать по двухслойной неявной схеме:

$$\frac{\partial u}{\partial t} \Big|_{t=t_s} = \frac{u_s - u_{s-1}}{\Delta t_s}$$

Тогда решение задачи сводится к нахождению решения на каждом временном слое путем решения задачи:

$$-\text{div}(\lambda(u_s) \text{grad}(u_s)) + \frac{\sigma}{\Delta t_s} u_s = f_s + \frac{\sigma}{\Delta t_s} u_{s-1},$$

где $u_s = u(x, t_s)$ – решение на s-м временном слое, $\Delta t_s = t_s - t_{s-1}$ – шаг по времени, $f_s = f(x, t_s)$ – значение правой части на s-м временном слое.

В результате конечноэлементной аппроксимации нелинейной начально-краевой задачи получается система не линейных уравнений:

$$A(q_s)q_s = b(q_s).$$

Метод простой итерации:

Компоненты матрицы A и вектора правой части b определяются с помощью локальных матриц жесткости и массы:

$$\widehat{G} = \frac{\lambda_s^1(u(\widehat{q}_s)) + \lambda_s^2(u(\widehat{q}_s))}{2h_k} \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}, \widehat{M}^t = \frac{\sigma h_k}{6\Delta t_s} \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$
$$\widehat{b}^c = \frac{h_k}{6} \begin{bmatrix} f^k(\widehat{x}_1) \\ f^k(\widehat{x}_2) \end{bmatrix} \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \widehat{b}^t = \frac{\sigma h_k}{6\Delta t_s} \begin{bmatrix} \widehat{q}_{s-1}^1 \\ \widehat{q}_{s-1}^2 \end{bmatrix} \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \widehat{b} = \widehat{b}^c + \widehat{b}^t,$$

где $\lambda_s^i(u(q_s))$ – значение коэффициента лямбда в i узле, q_{s-1}^i – значения весов с прошлого временного слоя.

Решение будем находить с помощью метода просто итерации, где для каждого временного слоя будет брать начальное приближение (будем брать значение вектора весов с предыдущего слоя), для него собираться СЛАУ и находится новый вектор весов. Так будем продолжать пока не выполнится условие:

$$\frac{\|A(q^k)q^k - b(q^k)\|}{\|b\|} < \varepsilon,$$

где ε – маленькое число, выбранное как требуемая точность решения нелинейной задачи, или в случае если итерационный процесс “упрется” в максимальное число итераций.

Метод Ньютона:

Компоненты локальных линейаризованной матрицы и вектора правой части будут находится с помощью формул:

$$\hat{A}_{ij}^L = \hat{A}_{ij}(\hat{q}^0) + v \sum_{r=1}^{\hat{n}} \frac{\partial \hat{A}_{ir}(\hat{q})}{\partial \hat{q}_j} \Big|_{\hat{q}=\hat{q}^0} \hat{q}_r^0$$

$$\hat{b}_i^L = \hat{b}_i(\hat{q}^0) + v \sum_{j=1}^{\hat{n}} \hat{q}_j^0 \sum_{r=1}^{\hat{n}} \frac{\partial \hat{A}_{ij}(\hat{q})}{\partial \hat{q}_r} \Big|_{\hat{q}=\hat{q}^0} \hat{q}_r^0$$

$\hat{q}^0$ на первой итерации – начальное приближение, на последующих в качестве него берется вектор $\hat{q}^{k-1}$, v – коэффициент демпфирования.

Так как работаем с нелинейность только коэффициента $\lambda(u)$, то формула, по которой считается производная от $\lambda(u)$ на области Ω_m , имеет вид:

$$\frac{\partial \lambda_{\Omega_m}(u(\hat{q}, \hat{x}_l))}{\partial \hat{q}_j} \Big|_{\hat{q}=\hat{q}^0} = \frac{\partial \lambda_{\Omega_m}(u)}{\partial u} \Big|_{u=u(\hat{q}, \hat{x}_l)} \frac{\partial u(\hat{q}, \hat{x}_l)}{\partial \hat{q}_j} \Big|_{\hat{q}=\hat{q}^0} = \frac{\partial \lambda_{\Omega_m}(u)}{\partial u} \Big|_{u=u(\hat{q}, \hat{x}_l)} \hat{\psi}_j(\hat{x}_l)$$

Итерационный процесс строится аналогично тому, как был построен в методе простой итерации. Каждый шаг с помощью $\hat{q}^{k-1}$ с помощью вышеуказанных формул генерируется линейаризованная система и находится новое приближение $\hat{q}^k$. После чего проверяется критерий останова по относительной невязка. При оценке относительной невязки используется матрица и вектор нелинеаризованной системы.

4 Тестирование

4.1 Равномерная сетка

$$4.1.1 u(x, t) = x^2; \lambda(u) = 1; \sigma = 1; f(x, t) = -2; u(x, 0) = x^2$$

Метод простой итерации

Time layer 0: t = 0.00000, countNodes = 4, ||u* - u||L2 = 0.0000e+00
Time layer 1: t = 1.50000, countNodes = 4, ht = 1.50000, countIteration = 0, ||u* - u||L2 = 0.0000e+00
Time layer 2: t = 3.00000, countNodes = 4, ht = 1.50000, countIteration = 0, ||u* - u||L2 = 0.0000e+00

Метод Ньютона

Time layer 0: t = 0.00000, countNodes = 4, ||u* - u||L2 = 0.0000e+00
Time layer 1: t = 1.50000, countNodes = 4, ht = 1.50000, countIteration = 0, ||u* - u||L2 = 9.1551e-16
Time layer 2: t = 3.00000, countNodes = 4, ht = 1.50000, countIteration = 0, ||u* - u||L2 = 9.1551e-16

$$4.1.2 u(x, t) = t^2; \lambda(u) = 1; \sigma = 1; f(x, t) = 2t; u(x, 0) = 0$$

Метод простой итерации

Time layer 0: t = 0.00000, countNodes = 4, ||u* - u||L2 = 0.0000e+00
Time layer 1: t = 1.50000, countNodes = 4, ht = 1.50000, countIteration = 0, ||u* - u||L2 = 1.3147e+00
Time layer 2: t = 3.00000, countNodes = 4, ht = 1.50000, countIteration = 0, ||u* - u||L2 = 1.8166e+00

Метод Ньютона

Time layer 0: t = 0.00000, countNodes = 4, ||u* - u||L2 = 0.0000e+00
Time layer 1: t = 1.50000, countNodes = 4, ht = 1.50000, countIteration = 0, ||u* - u||L2 = 1.3147e+00
Time layer 2: t = 3.00000, countNodes = 4, ht = 1.50000, countIteration = 0, ||u* - u||L2 = 1.8166e+00

Дробим сетку вдвое по t

Метод простой итерации

Time layer 0: t = 0.00000, countNodes = 4, ||u* - u||L2 = 0.0000e+00
Time layer 1: t = 0.60000, countNodes = 4, ht = 0.60000, countIteration = 0, ||u* - u||L2 = 3.3370e-01
Time layer 2: t = 1.20000, countNodes = 4, ht = 0.60000, countIteration = 0, ||u* - u||L2 = 5.3808e-01
Time layer 3: t = 1.80000, countNodes = 4, ht = 0.60000, countIteration = 0, ||u* - u||L2 = 6.6140e-01
Time layer 4: t = 2.40000, countNodes = 4, ht = 0.60000, countIteration = 0, ||u* - u||L2 = 7.3573e-01
Time layer 5: t = 3.00000, countNodes = 4, ht = 0.60000, countIteration = 0, ||u* - u||L2 = 7.8054e-01

Метод Ньютона

Time layer 0: t = 0.00000, countNodes = 4, ||u* - u||L2 = 0.0000e+00
Time layer 1: t = 0.60000, countNodes = 4, ht = 0.60000, countIteration = 0, ||u* - u||L2 = 3.3370e-01
Time layer 2: t = 1.20000, countNodes = 4, ht = 0.60000, countIteration = 0, ||u* - u||L2 = 5.3808e-01
Time layer 3: t = 1.80000, countNodes = 4, ht = 0.60000, countIteration = 0, ||u* - u||L2 = 6.6140e-01
Time layer 4: t = 2.40000, countNodes = 4, ht = 0.60000, countIteration = 0, ||u* - u||L2 = 7.3573e-01

Time layer 5: t = 3.00000, countNodes = 4, ht = 0.60000, countIteration = 0, ||u* - u||L2 = 7.8054e-01

$$4.1.3 \ u(x, t) = x^2; \lambda(u) = u; \sigma = 1; f(x, t) = -6x^2; u(x, 0) = x^2$$

Метод простой итерации

Time layer 0: t = 0.00000, countNodes = 4, ||u* - u||L2 = 0.0000e+00

Time layer 1: t = 1.50000, countNodes = 4, ht = 1.50000, countIteration = 0, ||u* - u||L2 = 2.0472e-15

Time layer 2: t = 3.00000, countNodes = 4, ht = 1.50000, countIteration = 0, ||u* - u||L2 = 3.2803e-15

Метод Ньютона

Time layer 0: t = 0.00000, countNodes = 4, ||u* - u||L2 = 0.0000e+00

Time layer 1: t = 1.50000, countNodes = 4, ht = 1.50000, countIteration = 0, ||u* - u||L2 = 9.9301e-16

Time layer 2: t = 3.00000, countNodes = 4, ht = 1.50000, countIteration = 0, ||u* - u||L2 = 8.9509e-16

$$4.1.4 \ u(x, t) = x^2 + t; \lambda(u) = u; \sigma = 1; f(x, t) = 1 - 6x^2 - 2t; u(x, 0) = x^2$$

Метод простой итерации

w=1.0

Time layer 0: t = 0.00000, countNodes = 4, ||u* - u||L2 = 0.0000e+00

Time layer 1: t = 1.50000, countNodes = 4, ht = 1.50000, countIteration = 0, ||u* - u||L2 = 1.8261e+00

Time layer 2: t = 3.00000, countNodes = 4, ht = 1.50000, countIteration = 0, ||u* - u||L2 = 2.1537e+00

w=0.1

Time layer 0: t = 0.00000, countNodes = 4, ||u* - u||L2 = 0.0000e+00

Time layer 1: t = 1.50000, countNodes = 4, ht = 1.50000, countIteration = 129, ||u* - u||L2 = 5.1751e-06

Time layer 2: t = 3.00000, countNodes = 4, ht = 1.50000, countIteration = 125, ||u* - u||L2 = 1.9551e-05

Метод Ньютона

Time layer 0: t = 0.00000, countNodes = 4, ||u* - u||L2 = 0.0000e+00

Time layer 1: t = 1.50000, countNodes = 4, ht = 1.50000, countIteration = 2, ||u* - u||L2 = 8.8818e-16

Time layer 2: t = 3.00000, countNodes = 4, ht = 1.50000, countIteration = 2, ||u* - u||L2 = 1.2561e-15

4.2 Неравномерная сетка

$$4.2.1 \ u(x, t) = x^2; \lambda(u) = 1; \sigma = 1; f(x, t) = -2; u(x, 0) = x^2$$

Метод простой итерации

Time layer 0: t = 0.00000, countNodes = 4, ||u* - u||L2 = 0.0000e+00

Time layer 1: t = 2.00000, countNodes = 4, ht = 2.00000, countIteration = 0, ||u* - u||L2 = 8.8818e-16

Time layer 2: t = 6.00000, countNodes = 4, ht = 4.00000, countIteration = 0, ||u* - u||L2 = 8.8818e-16

Time layer 3: t = 14.00000, countNodes = 4, ht = 8.00000, countIteration = 0, ||u* - u||L2 = 1.5177e-14

Метод Ньютона

Time layer 0: t = 0.00000, countNodes = 4, ||u* - u||L2 = 0.0000e+00

Time layer 1: t = 2.00000, countNodes = 4, ht = 2.00000, countIteration = 0, ||u* - u||L2 = 1.4239e-14

Time layer 2: t = 6.00000, countNodes = 4, ht = 4.00000, countIteration = 0, ||u* - u||L2 = 7.1054e-15

Time layer 3: t = 14.00000, countNodes = 4, ht = 8.00000, countIteration = 0, ||u* - u||L2 = 1.4321e-14

$$4.2.2 \ u(x, t) = x^2; \lambda(u) = \cos(u); \sigma = 1; \\ f(x, t) = 4x^2 \sin x^2 - 2 \cos x^2; u(x, 0) = x^2$$

Метод простой итерации

Time layer0:t =0.00000, countNodes = 4, ||u* - u||L2 = 0.0000e+00

Time layer1:t =2.00000, countNodes = 4, ht = 2.00000, countIteration = 10000, ||u* - u||L2 = 3.2214e+02

Time layer2:t =6.00000, countNodes = 4, ht = 4.00000, countIteration = 10000, ||u* - u||L2 = 9.4697e+02

Time layer3:t =14.00000, countNodes = 4, ht = 8.00000, countIteration = 10000, ||u* - u||L2 = 2.6950e+03

Метод Ньютона

Time layer 0: t = 0.00000, countNodes = 4, ||u* - u||L2 = 0.0000e+00

Time layer 1: t = 2.00000, countNodes = 4, ht = 2.00000, countIteration = 10000, ||u* - u||L2 = 4.9030e+03

Time layer 2: t = 6.00000, countNodes = 4, ht = 4.00000, countIteration = 10000, ||u* - u||L2 = 1.7586e+04

Time layer 3: t = 14.00000, countNodes = 4, ht = 8.00000, countIteration = 10000, ||u* - u||L2 = 2.1066e+04

5 Выводы

На равномерной сетке метод Простой итерации и метод Ньютона показывают близкие результаты, за счёт простоты задач, но в задаче 4.1.4 Метод Ньютона оказался точнее, за счёт использования линеаризации системы, так же видно, что при разных числах правдоподобия решение сходится с разным количеством итераций.

На не равномерной сетке получены удовлетворительные результаты. На задаче 4.2.2 решение получено с большой погрешностью за счёт того, что функция не представима в квадратичном базисе и сама функция стремительно изменяется на небольшом интервале.

6 Текст программы

6.3 Main.cpp

```
#include "Mesh.h"
#include "SimpleIteration.h"
#include "Newton.h"
#include <iostream>
#include <fstream>
#include <iomanip>
#include <sstream>
#include <string>
#include "LU.h"

std::string ToCommaString( double value, int precision, bool scientific = false )
{
    std::ostringstream stream;
    if ( scientific )
        stream << std::uppercase << std::scientific;
    else
        stream << std::fixed;

    stream << std::setprecision( precision ) << value;
    std::string text = stream.str( );
    for ( char& ch : text )
    {
        if ( ch == '.' )
            ch = ',';
    }
    return text;
}

template <typename Solver>
void WriteSolutionToFile( const Solver& solver, const std::string& fileName )
{
    std::ofstream file( fileName );
    if ( !file )
        return;

    file << "x\tq\tq*\tq - q*" << std::endl;

    Functions exactFunctions = solver.functions;

    for ( int i = 0; i < solver.n; i++ )
    {
        double x = 0.0;
        if ( i % 2 == 0 )
        {
            x = solver.mesh.meshX[i / 2];
```

```

    }
    else
    {
        int left = i / 2;
        x = ( solver.mesh.meshX[left] + solver.mesh.meshX[left + 1] ) / 2.0;
    }

    double exact = exactFunctions.u( x );
    double diff = solver.q[i] - exact;

    file
        << ToCommaString( x, 16 ) << "\t"
        << ToCommaString( solver.q[i], 16 ) << "\t"
        << ToCommaString( exact, 16 ) << "\t"
        << ToCommaString( diff, 2, true ) << std::endl;
    }
}

int main()
{
    int method, Id;

    std::cout << "Method (1 - Simple iteration, 2 - Newton): ";
    std::cin >> method;

    std::cout << "Function (1, 2, ...): ";
    std::cin >> Id;

    switch (method)
    {
    case 1:
    {
        SimpleIteration solver;
        solver.functions.id = Id;
        solver.useTimeDependent = true;

        solver.Input();
        solver.Solve();
        WriteSolutionToFile( solver, "out.txt" );

        break;
    }

    case 2:
    {
        Newton solver;
        solver.functions.id = Id;
        solver.useTimeDependent = true;

        solver.Input();
        solver.Solve();
        WriteSolutionToFile( solver, "out.txt" );

        break;
    }
    }

    return 0;
}

```

```

6.4 Basis.h
#pragma once
class Basis
{
public:

```

```

double basis1( double x );
double basis1Grad( double x );
double basis2( double x );
double basis2Grad( double x );
double basis3( double x );
double basis3Grad( double x );

double **Quadrature( );
};

6.5 Basis.cpp
#include "Basis.h"
#include <cmath>

double Basis::basis1( double x )
{
    return 2 * ( x - 0.5 ) * ( x - 1 );
}

double Basis::basis1Grad( double x )
{
    return 4 * x - 3;
}

double Basis::basis2( double x )
{
    return -4 * x * ( x - 1 );
}

double Basis::basis2Grad( double x )
{
    return -4 * ( 2 * x - 1 );
}

double Basis::basis3( double x )
{
    return 2 * x * ( x - 0.5 );
}

double Basis::basis3Grad( double x )
{
    return 4 * x - 1;
}

double **Basis::Quadrature( )
{
    double **res = new double *[3];
    double t1 = -std::sqrt( 15.0 ) / 5.0;
    double t2 = 0.0;
    double t3 = +std::sqrt( 15.0 ) / 5.0;
    double w1 = 5.0 / 9.0;
    double w2 = 8.0 / 9.0;
    double w3 = 5.0 / 9.0;

    res[0] = new double[2] {( t1 + 1 ) / 2, 0.5 * w1};
    res[1] = new double[2] {( t2 + 1 ) / 2, 0.5 * w2};
    res[2] = new double[2] {( t3 + 1 ) / 2, 0.5 * w3};
    return res;
}

6.6 Mesh.h
#pragma once
#include <vector>
class Mesh
{

```



```

public:
    std::vector<double> meshX;
    std::vector<double> meshT;
    int n_x, n_t;

    bool uniformX = true;
    bool uniformT = true;

    void InputMeshes( );

    void CalculateUniformMesh( std::vector<double> &mesh, double a, double b, int n);
    void CalculateNonUniformMesh( std::vector<double> &mesh, double a, double b, int n,
double q );
};

```

6.7 Mesh.cpp

```

#include "Mesh.h"
#include <fstream>
#include <iostream>

void Mesh::InputMeshes( )
{
    double a_x, b_x, q_x, a_t, b_t, q_t;
    std::ifstream f( "mesh.txt" );
    if ( !f )
    {
        std::cerr << "File open error" << std::endl;
        return;
    }

    f >> uniformX >> n_x >> a_x >> b_x;
    meshX.resize(n_x);
    if ( uniformX )
        CalculateUniformMesh( meshX, a_x, b_x, n_x );
    else
    {
        f >> q_x;
        CalculateNonUniformMesh( meshX, a_x, b_x, n_x, q_x );
    }

    f >> uniformT >> n_t >> a_t >> b_t;
    meshT.resize( n_t );
    if ( uniformT )
        CalculateUniformMesh( meshT, a_t, b_t, n_t );
    else
    {
        f >> q_t;
        CalculateNonUniformMesh( meshT, a_t, b_t, n_t, q_t );
    }
}

void Mesh::CalculateUniformMesh( std::vector<double> &mesh, double a, double b, int n
)
{
    double h = ( b - a ) / ( n - 1 );
    for ( int i = 0; i < n; i++ )
        mesh[i] = a + i * h;
}

void Mesh::CalculateNonUniformMesh( std::vector<double> &mesh, double a, double b, int
n, double q )
{
    double h = ( b - a ) * ( 1 - q ) / ( 1 - pow( q, n - 1 ) );
    double hi = h;
    mesh[0] = a;

```

```

double sum = 0;
for ( int i = 1; i < n - 1; i++ )
{
    sum += h * pow( q, i - 1 );
    mesh[i] = a + sum;
}
mesh[n-1] = b;
}

```

6.8 Func.h

```

#pragma once
#include <cmath>

class Functions
{
public:
    int id = 1;
    double currentTime = 0.0;

    double u(double x);
    double f(double x);
    double lambda(double u);
    double dlambda(double u);
    double sigma(double u);
    double sigmaTime(double t);
    double theta(double u);
    double beta(double u);
    double dbeta(double u);
    double uBeta(double u);
    double boundaryValue(int number, double x);
};

```

6.9 Func.cpp

```

#include "Func.h"
#include <numbers>

double Functions::u(double x)
{
    switch (id)
    {
    case 1:
        return x;

    case 2:
        return x * x;

    case 3:
        return currentTime * currentTime;

    case 4:
        return x * x;

    case 5:
        return x * x + currentTime;

    case 6:
        return x * x;

    default:
        return x;
    }
}

double Functions::f(double x)
{

```

```

switch (id)
{
case 1:
    return 0.0;

case 2:
    return -2.0;

case 3:
    return 2.0 * currentTime;

case 4:
    return -6.0 * x * x;

case 5:
    return 1.0 - 6.0 * x * x - 2.0 * currentTime;

case 6:
    return 4.0 * x * x * sin( x * x ) - 2.0 * cos( x * x );

default:
    return 0.0;
}
}

double Functions::lambda(double u)
{
    switch (id)
    {
    case 1:
    case 2:
    case 3:
        return 1.0;

    case 4:
        return u;
    case 5:
        return u;

    case 6:
        return cos( u );

    default:
        return 1.0;
    }
}

double Functions::dlambda(double u)
{
    switch (id)
    {
    case 1:
    case 2:
    case 3:
        return 0.0;

    case 4:
        return 1.0;
    case 5:
        return 1.0;

    case 6:
        return -sin( u );

    default:

```

```

        return 0.0;
    }
}

double Functions::sigma(double u)
{
    switch (id)
    {
        default:
            return 1.0;
    }
}

double Functions::sigmaTime(double t)
{
    switch (id)
    {
        default:
            return 1.0;
    }
}

double Functions::theta(double u)
{
    switch (id)
    {
        default:
            return 0.0;
    }
}

double Functions::beta(double u)
{
    switch (id)
    {
        default:
            return 0.0;
    }
}

double Functions::dbeta(double u)
{
    switch (id)
    {
        default:
            return 0.0;
    }
}

double Functions::uBeta(double u)
{
    switch (id)
    {
        default:
            return 0.0;
    }
}

double Functions::boundaryValue(int number, double x)
{
    switch (number)
    {
        case 1:
            return u(x);
        default:

```

```

        return 0.0;
    }
}

6.10 Matrix.h
#pragma once
#include <vector>
class Matrix
{
public:
    int n = 0;
    std::vector<double> di;
    std::vector<std::vector<double>> ggl;
    std::vector<std::vector<double>> ggu;
};

6.11 MatrixOperations.h
#pragma once
#include <vector>
#include "Matrix.h"
class MatrixOperations
{
public:
    Matrix MultMatScal( Matrix mat, double scal );
    std::vector<double> MultMatVec( Matrix mat, std::vector<double> vec );
    std::vector<double> MultVecScal( std::vector<double> vec, double scal );
    double DotProduct( std::vector<double> vec1, std::vector<double> vec2 );
    std::vector<double> AddVec( std::vector<double> vec1, std::vector<double> vec2 );
};

6.12 MatrixOperations.cpp
#include "MatrixOperations.h"

Matrix MatrixOperations::MultMatScal( Matrix mat, double scal )
{
    Matrix res = mat;
    for ( int i = 0; i < mat.n; i++ )
    {
        res.di[i] *= scal;
        for ( int j = 0; j < 2; j++ )
        {
            res.ggl[j][i] *= scal;
            res.ggu[j][i] *= scal;
        }
    }

    return res;
}

std::vector<double> MatrixOperations::MultMatVec( Matrix mat, std::vector<double> vec
)
{
    int n = mat.ggl[0].size();
    std::vector<double> y( mat.n, 0.0 );

    for ( int i = 0; i < mat.n; i++ )
    {
        // диагональ
        y[i] += mat.di[i] * vec[i];

        int maxI = std::max( n - i, 0 );
        // левая часть ( j < i )
        for ( int j = maxI; j < n; j++ )
        {
            y[j - n + i] += mat.ggu[i][j] * vec[i];

```

```

    }

    // правая часть (j > i)
    for ( int j = maxI; j < n; j++ )
    {
        y[i] += mat.ggl[i][j] * vec[j - n + i];
    }
}

return y;
}

std::vector<double> MatrixOperations::MultVecScal( std::vector<double> vec, double
scal )
{
    std::vector<double> res = vec;
    for ( int i = 0; i < vec.size(); i++ )
        res[i] *= scal;
    return res;
}

double MatrixOperations::DotProduct( std::vector<double> vec1, std::vector<double>
vec2 )
{
    double res = 0;
    for ( int i = 0; i < vec1.size( ); i++ )
        res += vec1[i]*vec2[i];
    return res;
}

std::vector<double> MatrixOperations::AddVec( std::vector<double> vec1,
std::vector<double> vec2 )
{
    std::vector<double> res = vec1;
    for ( int i = 0; i < vec1.size( ); i++ )
        res[i] += vec2[i];
    return res;
}

```

6.13 LU.h

```

#pragma once
#include "Matrix.h"
#include <vector>
class LU
{
public:
    LU( ) {}
    LU( Matrix _matrix, std::vector<double> _b )
    {
        matrix = _matrix;
        b = _b;
        y = std::vector<double>( matrix.n, 0.0 );
        q = std::vector<double>( matrix.n, 0.0 );
    }
    Matrix matrix;
    std::vector<double> b;
    std::vector<double> y;
    std::vector<double> q;

    void calcLU( );

    void calcY( );

    void calcQ( );

```

```
};
```

6.14 LU.cpp

```
#include "LU.h"
void LU::calcLU( ) {
    auto &ggl = matrix.ggl, &ggg = matrix.ggu;
    auto &di = matrix.di;
    const int ASize = di.size( ), K = 2;

    for ( int i = 0; i < ASize; i++ ) {
        double sumDi = 0;
        int j = i - K;
        for ( int jl = 0; jl < K; jl++, j++ ) {
            double sum1 = 0, sum2 = 0;
            if ( j < 0 ) continue;
            int mj = i - j;
            for ( int mi = 0; mi < jl; mi++, mj++ ) {
                sum1 += ggu[i][mi] * ggl[j][mj];
                sum2 += ggu[j][mj] * ggl[i][mi];
            }
            ggu[i][jl] -= sum1;
            ggl[i][jl] = ( ggl[i][jl] - sum2 ) / di[j];
            sumDi += ggu[i][jl] * ggl[i][jl];
        }
        di[i] -= sumDi;
    }
}

void LU::calcY( ) {
    auto &ggl = matrix.ggl;
    const int ASize = b.size( ), K = 2;
    y.resize( ASize );

    for ( int i = 0; i < ASize; i++ ) {
        double sum = 0;
        int j = i - K;
        for ( int jl = 0; jl < K; jl++, j++ ) {
            if ( j < 0 ) continue;
            sum += ggl[i][jl] * y[j];
        }
        y[i] = b[i] - sum;
    }
}

void LU::calcQ( ) {
    auto &ggg = matrix.ggu;
    auto &di = matrix.di;
    const int ASize = y.size( ), K = 2;

    for ( int i = ASize - 1; i >= 0; i-- ) {
        q[i] = y[i] / di[i];
        int j = i - K;
        for ( int jl = 0; jl < K; jl++, j++ ) {
            if ( j < 0 ) continue;
            y[j] -= ggu[i][jl] * q[i];
        }
    }
    y.clear( );
}
```

6.15 Element.h

```
#pragma once
#include "Basis.h"
#include <vector>
```

```

#include "Func.h"

class Element
{
public:
    Functions localFunctions;
    Functions* functions = &localFunctions;

    Element( ) {}
    Element( std::vector<double> coords_, int cond_, std::vector<double> q_,
std::vector<double> _f, std::vector<double> _lambdaNodes, Functions* functions_ =
nullptr )
    {
        coords = coords_;
        cond = cond_;
        localB.resize( 3 );
        f = _f;
        lambdaNodes = _lambdaNodes;
        q = q_;
        SetFunctions( functions_ );
    }

    void SetFunctions( Functions* functions_ );

    //параметры задачи
    double lambdaApprox( double x );
    double materialLambda( double u );
    double materialDLambda(double u);
    std::vector<double> lambdaNodes;
    std::vector<double> f;

    double theta( double u );
    double beta( double u );
    double dbeta(double u);
    double uBeta( double u );
    double uh( double x );
    double uhPrevTime( double x );

    std::vector<double> coords;
    int cond = 0;
    std::vector<double> q;
    std::vector<double> qPrevTime;
    bool useTimeDependent = false;
    double deltaT = 1.0;
    double currentTime = 0.0;
    std::vector<std::vector<double>> localMatrix = std::vector<std::vector<double>>( 3,
std::vector<double>( 3, 0.0 ) );
    std::vector<double> localB;

    Basis basis;

    void BuildLocalMatrix( );
    void BuildLocalB( );
    void BuildLocalMatrixNewton(std::vector<double> q_0);
    void BuildLocalBNewton(std::vector<double> q_0);
};

6.16 Element.cpp
#include "Element.h"

void Element::SetFunctions( Functions* functions_ )
{
    functions = functions_ != nullptr ? functions_ : &localFunctions;
}

```



```

double Element::lambdaApprox( double x )
{
    return lambdaNodes[0] * basis.basis1( x ) + lambdaNodes[1] * basis.basis2( x ) +
    lambdaNodes[2] * basis.basis3( x );
}

double Element::materialLambda( double u )
{
    return functions->lambda( u );
}

double Element::materialDLambda( double u )
{
    return functions->dlambda( u );
}

double Element::theta( double u )
{
    return functions->theta( u );
}

double Element::beta( double u )
{
    return functions->beta( u );
}

double Element::dbeta( double u )
{
    return functions->dbeta( u );
}

double Element::uBeta( double u )
{
    return functions->uBeta( u );
}

double Element::uh( double x )
{
    return q[0] * basis.basis1( x ) + q[1] * basis.basis2( x ) + q[2] * basis.basis3( x );
}

double Element::uhPrevTime( double x )
{
    return qPrevTime[0] * basis.basis1( x ) + qPrevTime[1] * basis.basis2( x ) +
    qPrevTime[2] * basis.basis3( x );
}

void Element::BuildLocalMatrix( )
{
    localMatrix = std::vector<std::vector<double>>( 3, std::vector<double>( 3, 0.0 ) );
    double **quadrature = basis.Quadrature( );
    double jacobian = coords[1] - coords[0];
    for ( int i = 0; i < 3; i++ )
    {
        double wj = quadrature[i][1] * jacobian;
        double psi1 = basis.basis1( quadrature[i][0] );
        double psi2 = basis.basis2( quadrature[i][0] );
        double psi3 = basis.basis3( quadrature[i][0] );
        double gradPsi1 = basis.basis1Grad( quadrature[i][0] ) / jacobian;
        double gradPsi2 = basis.basis2Grad( quadrature[i][0] ) / jacobian;
        double gradPsi3 = basis.basis3Grad( quadrature[i][0] ) / jacobian;
        double sigmaVal = useTimeDependent ? 0.0 : functions->sigma( uh(
quadrature[i][0] ) );
        double lambdas = lambdaApprox( quadrature[i][0] );
    }
}

```

```

        localMatrix[0][0] += wj * ( lambdas * gradPsi1 * gradPsi1 + sigmaVal * psi1 *
psi1 + beta( uh( quadrature[i][0] ) ) * psi1 * psi1 );
        localMatrix[0][1] += wj * ( lambdas * gradPsi1 * gradPsi2 + sigmaVal * psi1 *
psi2 + beta( uh( quadrature[i][0] ) ) * psi1 * psi2 );
        localMatrix[0][2] += wj * ( lambdas * gradPsi1 * gradPsi3 + sigmaVal * psi1 *
psi3 + beta( uh( quadrature[i][0] ) ) * psi1 * psi3 );
        localMatrix[1][0] += wj * ( lambdas * gradPsi2 * gradPsi1 + sigmaVal * psi2 *
psi1 + beta( uh( quadrature[i][0] ) ) * psi2 * psi1 );
        localMatrix[1][1] += wj * ( lambdas * gradPsi2 * gradPsi2 + sigmaVal * psi2 *
psi2 + beta( uh( quadrature[i][0] ) ) * psi2 * psi2 );
        localMatrix[1][2] += wj * ( lambdas * gradPsi2 * gradPsi3 + sigmaVal * psi2 *
psi3 + beta( uh( quadrature[i][0] ) ) * psi2 * psi3 );
        localMatrix[2][0] += wj * ( lambdas * gradPsi3 * gradPsi1 + sigmaVal * psi3 *
psi1 + beta( uh( quadrature[i][0] ) ) * psi3 * psi1 );
        localMatrix[2][1] += wj * ( lambdas * gradPsi3 * gradPsi2 + sigmaVal * psi3 *
psi2 + beta( uh( quadrature[i][0] ) ) * psi3 * psi2 );
        localMatrix[2][2] += wj * ( lambdas * gradPsi3 * gradPsi3 + sigmaVal * psi3 *
psi3 + beta( uh( quadrature[i][0] ) ) * psi3 * psi3 );
        if ( useTimeDependent )
        {
            double sigmaTimeVal = functions->sigmaTime( currentTime ) / deltaT;
            localMatrix[0][0] += wj * sigmaTimeVal * psi1 * psi1;
            localMatrix[0][1] += wj * sigmaTimeVal * psi1 * psi2;
            localMatrix[0][2] += wj * sigmaTimeVal * psi1 * psi3;
            localMatrix[1][0] += wj * sigmaTimeVal * psi2 * psi1;
            localMatrix[1][1] += wj * sigmaTimeVal * psi2 * psi2;
            localMatrix[1][2] += wj * sigmaTimeVal * psi2 * psi3;
            localMatrix[2][0] += wj * sigmaTimeVal * psi3 * psi1;
            localMatrix[2][1] += wj * sigmaTimeVal * psi3 * psi2;
            localMatrix[2][2] += wj * sigmaTimeVal * psi3 * psi3;
        }
    }
}

void Element::BuildLocalB( )
{
    localB = std::vector<double>( 3, 0.0 );
    double **quadrature = basis.Quadrature( );
    double jacobian = coords[1] - coords[0];
    for ( int i = 0; i < 3; i++ )
    {
        double wj = quadrature[i][1] * jacobian;
        double psi1 = basis.basis1( quadrature[i][0] );
        double psi2 = basis.basis2( quadrature[i][0] );
        double psi3 = basis.basis3( quadrature[i][0] );
        double fVal = f[0] * psi1 + f[1] * psi2 + f[2] * psi3;
        localB[0] += wj * ( fVal * psi1 + theta( uh( quadrature[i][0] ) ) * psi1 + beta(
uh( quadrature[i][0] ) ) * uBeta( uh( quadrature[i][0] ) ) * psi1 );
        localB[1] += wj * ( fVal * psi2 + theta( uh( quadrature[i][0] ) ) * psi2 + beta(
uh( quadrature[i][0] ) ) * uBeta( uh( quadrature[i][0] ) ) * psi2 );
        localB[2] += wj * ( fVal * psi3 + theta( uh( quadrature[i][0] ) ) * psi3 + beta(
uh( quadrature[i][0] ) ) * uBeta( uh( quadrature[i][0] ) ) * psi3 );
        if ( useTimeDependent )
        {
            double sigmaTimeVal = functions->sigmaTime( currentTime ) / deltaT;
            double uhPrev = uhPrevTime( quadrature[i][0] );
            localB[0] += wj * sigmaTimeVal * uhPrev * psi1;
            localB[1] += wj * sigmaTimeVal * uhPrev * psi2;
            localB[2] += wj * sigmaTimeVal * uhPrev * psi3;
        }
    }
}

void Element::BuildLocalMatrixNewton( std::vector<double> q_0 )
{

```

```

localMatrix = std::vector<std::vector<double>>>( 3, std::vector<double>( 3, 0.0 ) );

double** quadrature = basis.Quadrature();
double jacobian = coords[1] - coords[0];
for ( int g = 0; g < 3; g++ )
{
    std::vector<double> psi( 3, 0.0 );
    std::vector<double> gradPsi( 3, 0.0 );
    double wj = quadrature[g][1] * jacobian;

    psi[0] = basis.basis1( quadrature[g][0] );
    psi[1] = basis.basis2( quadrature[g][0] );
    psi[2] = basis.basis3( quadrature[g][0] );

    gradPsi[0] = basis.basis1Grad( quadrature[g][0] ) / jacobian;
    gradPsi[1] = basis.basis2Grad( quadrature[g][0] ) / jacobian;
    gradPsi[2] = basis.basis3Grad( quadrature[g][0] ) / jacobian;

    double uhValue = uh( quadrature[g][0] );
    double sigmaVal = useTimeDependent ? 0.0 : functions->sigma( uhValue );
    double lambdaValue = lambdaApprox( quadrature[g][0] );
    double betaValue = beta( uhValue );
    double sigmaTimeValue = useTimeDependent ? functions->sigmaTime( currentTime ) /
deltaT : 0.0;

    double uhGradValue = 0.0;
    for ( int r = 0; r < 3; r++ )
    {
        uhGradValue += q_0[r] * gradPsi[r];
    }

    for ( int i = 0; i < 3; i++ )
    {
        for ( int j = 0; j < 3; j++ )
        {
            double A_ij = lambdaValue * gradPsi[i] * gradPsi[j] +
                sigmaVal * psi[i] * psi[j] +
                betaValue * psi[i] * psi[j] +
                sigmaTimeValue * psi[i] * psi[j];

            double dA_q = materialDLambda( q_0[j] ) * psi[j] * uhGradValue *
gradPsi[i];
            localMatrix[i][j] += wj * ( A_ij + dA_q );
        }
    }
}

void Element::BuildLocalBNewton( std::vector<double> q_0 )
{
    localB = std::vector<double>( 3, 0.0 );
    double** quadrature = basis.Quadrature();
    double jacobian = coords[1] - coords[0];
    for ( int g = 0; g < 3; g++ )
    {
        std::vector<double> psi( 3, 0.0 );
        std::vector<double> gradPsi( 3, 0.0 );
        double wj = quadrature[g][1] * jacobian;

        psi[0] = basis.basis1( quadrature[g][0] );
        psi[1] = basis.basis2( quadrature[g][0] );
        psi[2] = basis.basis3( quadrature[g][0] );

        gradPsi[0] = basis.basis1Grad( quadrature[g][0] ) / jacobian;
        gradPsi[1] = basis.basis2Grad( quadrature[g][0] ) / jacobian;

```

```

gradPsi[2] = basis.basis3Grad( quadrature[g][0] ) / jacobian;

double uhValue = uh( quadrature[g][0] );
double thetaValue = theta( uhValue );
double betaValue = beta( uhValue );
double uBetaValue = uBeta( uhValue );
double sigmaTimeValue = useTimeDependent ? functions->sigmaTime( currentTime ) /
deltaT : 0.0;
double uhPrev = useTimeDependent ? uhPrevTime( quadrature[g][0] ) : 0.0;
double fValue = f[0] * psi[0] + f[1] * psi[1] + f[2] * psi[2];

double uhGradValue = 0.0;
double dLambdaValue = 0.0;
for ( int r = 0; r < 3; r++ )
{
    uhGradValue += q_0[r] * gradPsi[r];
    dLambdaValue += materialDLambda( q_0[r] ) * q_0[r] * psi[r];
}

for ( int i = 0; i < 3; i++ )
{
    double B_i = fValue * psi[i] +
        thetaValue * psi[i] +
        betaValue * uBetaValue * psi[i] +
        sigmaTimeValue * uhPrev * psi[i];

    double dA_q = dLambdaValue * uhGradValue * gradPsi[i];
    localB[i] += wj * ( B_i + dA_q );
}
}
}

```

6.17 SimpleIteration.h

```

#pragma once
#include "Basis.h"
#include "Mesh.h";
#include "Element.h"
#include "Matrix.h"
#include "LU.h"
#include "MatrixOperations.h"
#include "Func.h"

class SimpleIteration
{
public:
    double eps = 1e-16;
    int maxIter = 100;
    int elemCount = 0;
    int n = 0;

    Functions functions;

    //Крайевые условия
    int boundaryConditions[2];
    double Condition(int number, double x );
    void Condition1( );

    std::vector<double> q;
    std::vector<double> q0Ld;
    std::vector<double> qPrevTime;

    //матрица
    Matrix matrix;

    //вектор правой части

```

```

std::vector<double> b;
std::vector<double> b0;

double deltaT;
bool useTimeDependent = false;
int timeLayer = 0;

double w = 0.01;

std::vector<Element> elements;
void Input( );
void UpdateLayerData( );
void BuildMatrix( );
void BuildB( );
void SolveIter( );
void Solve( );
double CalcResidual( );
double CalcL2Error( );
void PrintTimeLayerInfo( int layerIndex, int iterationCount );
std::vector<double> lambdaNodes;

Mesh mesh;
Basis basis;
LU lu;
MatrixOperations ops;
};

```

6.18 SimpleIteration.cpp

```

#include "SimpleIteration.h"
#include <iostream>
#include <fstream>
#include <iomanip>
#include <cmath>

double SimpleIteration::Condition( int number, double x )
{
    return functions.boundaryValue( number, x );
}

void SimpleIteration::Condition1( )
{
    if ( elements[0].cond == 1 )
    {
        matrix.di[0] = 1;
        b[0] = Condition( 1, 0 );
        for ( int i = 0; i < 2; i++ )
        {
            matrix.ggu[1][1] = 0;
            matrix.ggu[2][0] = 0;
        }
    }
    if ( elements[elemCount - 1].cond == 1 )
    {
        matrix.di[n - 1] = 1;
        b[n - 1] = Condition( 1, mesh.meshX[elemCount] );
        for ( int i = 0; i < 2; i++ )
        {
            matrix.ggl[n - 1][0] = 0;
            matrix.ggl[n - 1][1] = 0;
        }
    }
}

void SimpleIteration::Input( )
{

```

```

mesh.InputMeshes( );
std::ifstream file( "params.txt" );
if ( !file )
{
    std::cerr << "File open error" << std::endl;
    return;
}
file >> eps >> maxIter >> boundaryConditions[0] >> boundaryConditions[1];

elemCount = mesh.n_x - 1;
n = 3 * elemCount - elemCount + 1;
matrix.n = n;
matrix.ggl.resize( n );
matrix.ggu.resize( n );
for ( int i = 0; i < n; i++ )
{
    matrix.ggl[i] = std::vector<double>( 2, 0.0 );
    matrix.ggu[i] = std::vector<double>( 2, 0.0 );
}
matrix.di = std::vector<double>( n, 0.0 );

b.resize( n );
lambdaNodes.resize( n );
b0 = b;

std::vector<double> coords;
elements.resize( elemCount );
q = std::vector<double>( n, 0.0 );
if ( useTimeDependent )
{
    functions.currentTime = mesh.meshT[0];
    for ( int i = 0; i <= elemCount; i++ )
    {
        q[i * 2] = functions.u( mesh.meshX[i] );
        if ( i < elemCount )
            q[i * 2 + 1] = functions.u( ( mesh.meshX[i] + mesh.meshX[i + 1] ) / 2.0 );
    }
}
q[0] = Condition( 1, 0 );
q[n - 1] = Condition( 1, mesh.meshX[elemCount] );
qPrevTime = q;
std::vector<double> localB( 3, 0.0 );
std::vector<double> localLambdaNodes( 3, 0.0 );
for ( int i = 0; i < elemCount; i++ )
{
    coords = { mesh.meshX[i], mesh.meshX[i + 1] };
    for ( int j = 0; j < 3; j++ )
    {
        if ( i == 0 )
        {
            localB[j] = b[3 * i + j];
            localLambdaNodes[j] = lambdaNodes[3 * i + j];
        }
        else
        {
            localB[j] = b[2 * i + j];
            localLambdaNodes[j] = lambdaNodes[2 * i + j];
        }
    }
    elements[i] = Element( coords, 0, q, localB, localLambdaNodes, &functions );
}
elements[0].cond = boundaryConditions[0];
elements[elemCount - 1].cond = boundaryConditions[1];
UpdateLayerData( );

```

```

}

void SimpleIteration::BuildMatrix( )
{
    matrix.di = std::vector<double>( n, 0.0 );
    for ( int i = 0; i < n; i++ )
    {
        matrix.ggl[i] = std::vector<double>( 2, 0.0 );
        matrix.ggu[i] = std::vector<double>( 2, 0.0 );
    }

    for ( int i = 0; i < elemCount; i++ )
    {
        elements[i].useTimeDependent = useTimeDependent;
        elements[i].deltaT = deltaT;
        elements[i].currentTime = mesh.meshT[timeLayer];
        elements[i].BuildLocalMatrix( );
        for ( int j = 0; j < 3; j++ )
        {
            if ( i == 0 )
                matrix.di[3 * i + j] += elements[i].localMatrix[j][j];
            else
                matrix.di[2 * i + j] += elements[i].localMatrix[j][j];
        }
        matrix.ggl[2 * i + 1][1] += elements[i].localMatrix[1][0];
        matrix.ggl[2 * i + 2][0] += elements[i].localMatrix[2][0];
        matrix.ggl[2 * i + 2][1] += elements[i].localMatrix[2][1];

        matrix.ggu[2 * i + 1][1] += elements[i].localMatrix[0][1];
        matrix.ggu[2 * i + 2][0] += elements[i].localMatrix[0][2];
        matrix.ggu[2 * i + 2][1] += elements[i].localMatrix[1][2];
    }
}

void SimpleIteration::BuildB( )
{
    b = std::vector<double>( n, 0.0 );
    for ( int i = 0; i < elemCount; i++ )
    {
        elements[i].useTimeDependent = useTimeDependent;
        elements[i].deltaT = deltaT;
        elements[i].currentTime = mesh.meshT[timeLayer];
        elements[i].BuildLocalB( );
        for ( int j = 0; j < 3; j++ )
        {
            if ( i == 0 )
                b[3 * i + j] += elements[i].localB[j];
            else
                b[2 * i + j] += elements[i].localB[j];
        }
    }
}

void SimpleIteration::SolveIter( )
{
    qOld = q;
    UpdateLayerData( );
    BuildMatrix( );
    BuildB( );
    Condition1( );
    lu = LU( matrix, b );
    lu.calcLU( );
    lu.calcY( );
}

```

```

    lu.calcQ( );
    q = ops.AddVec( ops.MultVecScal( lu.q, w ), ops.MultVecScal( qOld, 1 - w ) );
    std::vector<double> localQ( 3, 0.0 );
    for ( int i = 0; i < elemCount; i++ )
    {
        for ( int j = 0; j < 3; j++ )
        {
            if ( i == 0 )
            {
                localQ[j] = q[3 * i + j];
            }
            else
            {
                localQ[j] = q[2 * i + j];
            }
        }
        elements[i].q = localQ;
    }
}

void SimpleIteration::Solve( )
{
    if ( !useTimeDependent )
    {
        int k = 0;
        SolveIter( );
        double residual = CalcResidual( );

        while ( residual > eps && k < maxIter )
        {
            k++;
            residual = CalcResidual( );
            std::cout << k << "\tresidual: " << residual << std::endl;
            for ( int i = 0; i < elemCount; i++ )
                for ( int j = 0; j < 3; j++ )
                {
                    if ( i == 0 )
                        elements[i].q[j] = q[3 * i + j];
                    else
                        elements[i].q[j] = q[2 * i + j];
                }
            SolveIter( );
        }

        return;
    }

    timeLayer = 0;
    functions.currentTime = mesh.meshT[0];
    PrintTimeLayerInfo( 0, 0 );

    for ( int s = 1; s < mesh.n_t; s++ )
    {
        timeLayer = s;
        deltaT = mesh.meshT[s] - mesh.meshT[s - 1];
        qPrevTime = q;
        q = qPrevTime;

        for ( int i = 0; i < elemCount; i++ )
        {
            std::vector<double> localQ( 3, 0.0 );
            std::vector<double> localPrevQ( 3, 0.0 );
            for ( int j = 0; j < 3; j++ )
            {

```



```

        if ( i == 0 )
        {
            localQ[j] = q[3 * i + j];
            localPrevQ[j] = qPrevTime[3 * i + j];
        }
        else
        {
            localQ[j] = q[2 * i + j];
            localPrevQ[j] = qPrevTime[2 * i + j];
        }
    }
    elements[i].q = localQ;
    elements[i].qPrevTime = localPrevQ;
}

int k = 0;
SolveIter( );
double residual = CalcResidual( );

while ( residual > eps && k < maxIter )
{
    k++;
    residual = CalcResidual( );
    SolveIter( );
}

functions.currentTime = mesh.meshT[s];
PrintTimeLayerInfo( s, k );
}
}

void SimpleIteration::UpdateLayerData( )
{
    functions.currentTime = mesh.meshT[timeLayer];
    b.assign( n, 0.0 );
    lambdaNodes.assign( n, 0.0 );

    for ( int i = 0; i <= elemCount; i++ )
    {
        b[i * 2] = functions.f( mesh.meshX[i] );
        lambdaNodes[i * 2] = functions.lambda( q[i * 2] );
        if ( i < elemCount )
        {
            double middle = ( mesh.meshX[i] + mesh.meshX[i + 1] ) / 2.0;
            b[i * 2 + 1] = functions.f( middle );
            lambdaNodes[i * 2 + 1] = functions.lambda( q[i * 2 + 1] );
        }
    }

    for ( int i = 0; i < elemCount; i++ )
    {
        for ( int j = 0; j < 3; j++ )
        {
            if ( i == 0 )
            {
                elements[i].f[j] = b[3 * i + j];
                elements[i].lambdaNodes[j] = lambdaNodes[3 * i + j];
            }
            else
            {
                elements[i].f[j] = b[2 * i + j];
                elements[i].lambdaNodes[j] = lambdaNodes[2 * i + j];
            }
        }
    }
}

```

```

}

double SimpleIteration::CalcResidual( )
{
    Matrix A_nonlin;
    A_nonlin.n = n;
    A_nonlin.di.assign( n, 0.0 );
    A_nonlin.ggl.assign( n, std::vector<double>( 2, 0.0 ) );
    A_nonlin.ggu.assign( n, std::vector<double>( 2, 0.0 ) );

    std::vector<double> b_nonlin( n, 0.0 );

    for ( int i = 0; i < elemCount; i++ )
    {
        elements[i].useTimeDependent = useTimeDependent;
        elements[i].deltaT = deltaT;
        elements[i].currentTime = mesh.meshT[timeLayer];
        elements[i].BuildLocalMatrix( );
        elements[i].BuildLocalB( );

        for ( int j = 0; j < 3; j++ )
        {
            if ( i == 0 )
                A_nonlin.di[3 * i + j] += elements[i].localMatrix[j][j];
            else
                A_nonlin.di[2 * i + j] += elements[i].localMatrix[j][j];
        }

        A_nonlin.ggl[2 * i + 1][1] += elements[i].localMatrix[1][0];
        A_nonlin.ggl[2 * i + 2][0] += elements[i].localMatrix[2][0];
        A_nonlin.ggl[2 * i + 2][1] += elements[i].localMatrix[2][1];

        A_nonlin.ggu[2 * i + 1][1] += elements[i].localMatrix[0][1];
        A_nonlin.ggu[2 * i + 2][0] += elements[i].localMatrix[0][2];
        A_nonlin.ggu[2 * i + 2][1] += elements[i].localMatrix[1][2];

        for ( int j = 0; j < 3; j++ )
        {
            if ( i == 0 )
                b_nonlin[3 * i + j] += elements[i].localB[j];
            else
                b_nonlin[2 * i + j] += elements[i].localB[j];
        }
    }

    if ( elements[0].cond == 1 )
    {
        A_nonlin.di[0] = 1;
        b_nonlin[0] = Condition( 1, 0 );
        A_nonlin.ggu[1][1] = 0;
        A_nonlin.ggu[2][0] = 0;
    }
    if ( elements[elemCount - 1].cond == 1 )
    {
        A_nonlin.di[n - 1] = 1;
        b_nonlin[n - 1] = Condition( 1, mesh.meshX[elemCount] );
        A_nonlin.ggl[n - 1][0] = 0;
        A_nonlin.ggl[n - 1][1] = 0;
    }

    std::vector<double> r = ops.AddVec(
        ops.MultMatVec( A_nonlin, q ),
        ops.MultVecScal( b_nonlin, -1 )
    );
}

```

```

    return ops.DotProduct( r, r ) / ops.DotProduct( b_nonlin, b_nonlin );
}

double SimpleIteration::CalcL2Error( )
{
    double sum = 0.0;
    for ( int i = 0; i < mesh.n_x; i++ )
    {
        double diff = q[2 * i] - functions.u( mesh.meshX[i] );
        sum += diff * diff;
    }
    return std::sqrt( sum );
}

void SimpleIteration::PrintTimeLayerInfo( int layerIndex, int iterationCount )
{
    double error = CalcL2Error( );
    std::streamsize oldPrecision = std::cout.precision();
    std::ios::fmtflags oldFlags = std::cout.flags();

    std::cout << std::fixed << std::setprecision( 5 )
        << "Time layer " << layerIndex << ": t = " << mesh.meshT[layerIndex]
        << ", countNodes = " << mesh.n_x;

    if ( layerIndex > 0 )
    {
        std::cout << ", ht = " << ( mesh.meshT[layerIndex] - mesh.meshT[layerIndex - 1]
    )
        << ", countIteration = " << iterationCount;
    }

    std::cout << std::scientific << std::setprecision( 4 )
        << ", ||u* - u||L2 = " << error << std::endl;

    std::cout.flags( oldFlags );
    std::cout.precision( oldPrecision );
}

```

6.19 Newton.h

```

#pragma once
#include "Basis.h"
#include "Mesh.h";
#include "Element.h"
#include "Matrix.h"
#include "LU.h"
#include "MatrixOperations.h"
#include "Func.h"

class Newton
{
public:
    double eps = 1e-16;
    int maxIter = 100;
    int elemCount = 0;
    int n = 0;

    Functions functions;

    //Краевые условия
    int boundaryConditions[2];
    double Condition(int number, double x);
    void Condition1();

    std::vector<double> q;
    std::vector<double> q0Ld;
}

```

```

std::vector<double> qPrevTime;

//матрица
Matrix matrix;

//вектор правой части
std::vector<double> b;
std::vector<double> b0;

double deltaT;
bool useTimeDependent = false;
int timeLayer = 0;

double w = 1.0;

std::vector<Element> elements;
void Input();
void UpdateLayerData();
void BuildMatrix(std::vector<double> qOld);
void BuildB(std::vector<double> qOld);
void SolveIter();
void Solve();
double CalcResidual();
double CalcL2Error();
void PrintTimeLayerInfo(int layerIndex, int iterationCount);
std::vector<double> lambdaNodes;

Mesh mesh;
Basis basis;
LU lu;
MatrixOperations ops;
};

```

6.20 Newton.cpp

```

#include "Newton.h"
#include <iostream>
#include <fstream>
#include <iomanip>
#include <cmath>

double Newton::Condition(int number, double x)
{
    return functions.boundaryValue(number, x);
}

void Newton::Condition1()
{
    if (elements[0].cond == 1)
    {
        matrix.di[0] = 1;
        b[0] = Condition(1, 0);
        for (int i = 0; i < 2; i++)
        {
            matrix.ggu[1][1] = 0;
            matrix.ggu[2][0] = 0;
        }
    }
    if (elements[elemCount - 1].cond == 1)
    {
        matrix.di[n - 1] = 1;
        b[n - 1] = Condition(1, mesh.meshX[elemCount]);
        for (int i = 0; i < 2; i++)
        {
            matrix.ggl[n - 1][0] = 0;
            matrix.ggl[n - 1][1] = 0;
        }
    }
}

```

```

    }
}

void Newton::Input()
{
    mesh.InputMeshes();
    std::ifstream file("params.txt");
    if (!file)
    {
        std::cerr << "File open error" << std::endl;
        return;
    }
    file >> eps >> maxIter >> boundaryConditions[0] >> boundaryConditions[1];

    elemCount = mesh.n_x - 1;
    n = 3 * elemCount - elemCount + 1;
    matrix.n = n;
    matrix.ggl.resize(n);
    matrix.ggu.resize(n);
    for (int i = 0; i < n; i++)
    {
        matrix.ggl[i] = std::vector<double>(2, 0.0);
        matrix.ggu[i] = std::vector<double>(2, 0.0);
    }
    matrix.di = std::vector<double>(n, 0.0);

    b.resize(n);
    lambdaNodes.resize(n);
    b0 = b;

    std::vector<double> coords;
    elements.resize(elemCount);
    q = std::vector<double>(n, 0.0);
    if (useTimeDependent)
    {
        functions.currentTime = mesh.meshT[0];
        for (int i = 0; i <= elemCount; i++)
        {
            q[i * 2] = functions.u(mesh.meshX[i]);
            if (i < elemCount)
                q[i * 2 + 1] = functions.u((mesh.meshX[i] + mesh.meshX[i + 1]) / 2.0);
        }
    }
    q[0] = Condition(1, 0);
    q[n - 1] = Condition(1, mesh.meshX[elemCount]);
    qPrevTime = q;
    std::vector<double> localB(3, 0.0);
    std::vector<double> localLambdaNodes(3, 0.0);
    for (int i = 0; i < elemCount; i++)
    {
        coords = { mesh.meshX[i], mesh.meshX[i + 1] };
        for (int j = 0; j < 3; j++)
        {
            if (i == 0)
            {
                localB[j] = b[3 * i + j];
                localLambdaNodes[j] = lambdaNodes[3 * i + j];
            }
            else
            {
                localB[j] = b[2 * i + j];
                localLambdaNodes[j] = lambdaNodes[2 * i + j];
            }
        }
    }
}

```

```

    }
    elements[i] = Element(coords, 0, q, localB, localLambdaNodes, &functions);
}
elements[0].cond = boundaryConditions[0];
elements[elemCount - 1].cond = boundaryConditions[1];
UpdateLayerData();

}

void Newton::BuildMatrix(std::vector<double> qOld)
{
    matrix.di = std::vector<double>(n, 0.0);
    for (int i = 0; i < n; i++)
    {
        matrix.ggl[i] = std::vector<double>(2, 0.0);
        matrix.ggu[i] = std::vector<double>(2, 0.0);
    }

    for (int i = 0; i < elemCount; i++)
    {
        std::vector<double> localQOld(3, 0.0);
        for (int j = 0; j < 3; j++)
        {
            if (i == 0)
                localQOld[j] = qOld[3 * i + j];
            else
                localQOld[j] = qOld[2 * i + j];
        }
        elements[i].useTimeDependent = useTimeDependent;
        elements[i].deltaT = deltaT;
        elements[i].currentTime = mesh.meshT[timeLayer];
        elements[i].BuildLocalMatrixNewton(localQOld);
        for (int j = 0; j < 3; j++)
        {
            if (i == 0)
                matrix.di[3 * i + j] += elements[i].localMatrix[j][j];
            else
                matrix.di[2 * i + j] += elements[i].localMatrix[j][j];
        }
        matrix.ggl[2 * i + 1][1] += elements[i].localMatrix[1][0];
        matrix.ggl[2 * i + 2][0] += elements[i].localMatrix[2][0];
        matrix.ggl[2 * i + 2][1] += elements[i].localMatrix[2][1];

        matrix.ggu[2 * i + 1][1] += elements[i].localMatrix[0][1];
        matrix.ggu[2 * i + 2][0] += elements[i].localMatrix[0][2];
        matrix.ggu[2 * i + 2][1] += elements[i].localMatrix[1][2];
    }
}

void Newton::BuildB(std::vector<double> qOld)
{
    b = std::vector<double>(n, 0.0);
    for (int i = 0; i < elemCount; i++)
    {
        std::vector<double> localQOld(3, 0.0);
        for (int j = 0; j < 3; j++)
        {
            if (i == 0)
                localQOld[j] = qOld[3 * i + j];
            else
                localQOld[j] = qOld[2 * i + j];
        }
        elements[i].useTimeDependent = useTimeDependent;
        elements[i].deltaT = deltaT;
    }
}

```

```

        elements[i].currentTime = mesh.meshT[timeLayer];
        elements[i].BuildLocalBNewton(localQOld);
        for (int j = 0; j < 3; j++)
        {
            if (i == 0)
                b[3 * i + j] += elements[i].localB[j];
            else
                b[2 * i + j] += elements[i].localB[j];
        }
    }
}

void Newton::SolveIter()
{
    qOld = q;
    UpdateLayerData();
    BuildMatrix(qOld);
    BuildB(qOld);
    Condition1();
    lu = LU(matrix, b);
    lu.calcLU();
    lu.calcY();
    lu.calcQ();
    q = ops.AddVec(ops.MultVecScal(lu.q, w), ops.MultVecScal(qOld, 1 - w));
    std::vector<double> localQ(3, 0.0);
    for (int i = 0; i < elemCount; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            if (i == 0)
            {
                localQ[j] = q[3 * i + j];
            }
            else
            {
                localQ[j] = q[2 * i + j];
            }
        }
        elements[i].q = localQ;
    }
}

void Newton::Solve()
{
    if (!useTimeDependent)
    {
        int k = 0;
        SolveIter();
        double residual = CalcResidual();

        while (residual > eps && k < maxIter)
        {
            k++;
            residual = CalcResidual();
            std::cout << k << "\tresidual: " << residual << std::endl;
            for (int i = 0; i < elemCount; i++)
                for (int j = 0; j < 3; j++)
                {
                    if (i == 0)
                        elements[i].q[j] = q[3 * i + j];
                    else
                        elements[i].q[j] = q[2 * i + j];
                }
            SolveIter();
        }
    }
}

```

```

    }

    return;
}

timeLayer = 0;
functions.currentTime = mesh.meshT[0];
PrintTimeLayerInfo(0, 0);

for (int s = 1; s < mesh.n_t; s++)
{
    timeLayer = s;
    deltaT = mesh.meshT[s] - mesh.meshT[s - 1];
    qPrevTime = q;
    q = qPrevTime;

    for (int i = 0; i < elemCount; i++)
    {
        std::vector<double> localQ(3, 0.0);
        std::vector<double> localPrevQ(3, 0.0);
        for (int j = 0; j < 3; j++)
        {
            if (i == 0)
            {
                localQ[j] = q[3 * i + j];
                localPrevQ[j] = qPrevTime[3 * i + j];
            }
            else
            {
                localQ[j] = q[2 * i + j];
                localPrevQ[j] = qPrevTime[2 * i + j];
            }
        }
        elements[i].q = localQ;
        elements[i].qPrevTime = localPrevQ;
    }

    int k = 0;
    SolveIter();
    double residual = CalcResidual();

    while (residual > eps && k < maxIter)
    {
        k++;
        residual = CalcResidual();
        SolveIter();
    }

    functions.currentTime = mesh.meshT[s];
    PrintTimeLayerInfo(s, k);
}
}

void Newton::UpdateLayerData()
{
    functions.currentTime = mesh.meshT[timeLayer];
    b.assign(n, 0.0);
    lambdaNodes.assign(n, 0.0);

    for (int i = 0; i <= elemCount; i++)
    {
        b[i * 2] = functions.f(mesh.meshX[i]);
        lambdaNodes[i * 2] = functions.lambda(q[i * 2]);
        if (i < elemCount)
        {

```



```

        double middle = (mesh.meshX[i] + mesh.meshX[i + 1]) / 2.0;
        b[i * 2 + 1] = functions.f(middle);
        lambdaNodes[i * 2 + 1] = functions.lambda(q[i * 2 + 1]);
    }
}

for (int i = 0; i < elemCount; i++)
{
    for (int j = 0; j < 3; j++)
    {
        if (i == 0)
        {
            elements[i].f[j] = b[3 * i + j];
            elements[i].lambdaNodes[j] = lambdaNodes[3 * i + j];
        }
        else
        {
            elements[i].f[j] = b[2 * i + j];
            elements[i].lambdaNodes[j] = lambdaNodes[2 * i + j];
        }
    }
}

double Newton::CalcResidual()
{
    Matrix A_nonlin;
    A_nonlin.n = n;
    A_nonlin.di.assign(n, 0.0);
    A_nonlin.ggl.assign(n, std::vector<double>(2, 0.0));
    A_nonlin.ggu.assign(n, std::vector<double>(2, 0.0));

    std::vector<double> b_nonlin(n, 0.0);

    for (int i = 0; i < elemCount; i++)
    {
        elements[i].useTimeDependent = useTimeDependent;
        elements[i].deltaT = deltaT;
        elements[i].currentTime = mesh.meshT[timeLayer];
        elements[i].BuildLocalMatrix();
        elements[i].BuildLocalB();

        for (int j = 0; j < 3; j++)
        {
            if (i == 0)
                A_nonlin.di[3 * i + j] += elements[i].localMatrix[j][j];
            else
                A_nonlin.di[2 * i + j] += elements[i].localMatrix[j][j];
        }

        A_nonlin.ggl[2 * i + 1][1] += elements[i].localMatrix[1][0];
        A_nonlin.ggl[2 * i + 2][0] += elements[i].localMatrix[2][0];
        A_nonlin.ggl[2 * i + 2][1] += elements[i].localMatrix[2][1];

        A_nonlin.ggu[2 * i + 1][1] += elements[i].localMatrix[0][1];
        A_nonlin.ggu[2 * i + 2][0] += elements[i].localMatrix[0][2];
        A_nonlin.ggu[2 * i + 2][1] += elements[i].localMatrix[1][2];

        for (int j = 0; j < 3; j++)
        {
            if (i == 0)
                b_nonlin[3 * i + j] += elements[i].localB[j];
            else
                b_nonlin[2 * i + j] += elements[i].localB[j];
        }
    }
}

```

```

    }

    if (elements[0].cond == 1)
    {
        A_nonlin.di[0] = 1;
        b_nonlin[0] = Condition(1, 0);
        A_nonlin.ggu[1][1] = 0;
        A_nonlin.ggu[2][0] = 0;
    }
    if (elements[elemCount - 1].cond == 1)
    {
        A_nonlin.di[n - 1] = 1;
        b_nonlin[n - 1] = Condition(1, mesh.meshX[elemCount]);
        A_nonlin.ggl[n - 1][0] = 0;
        A_nonlin.ggl[n - 1][1] = 0;
    }

    std::vector<double> r = ops.AddVec(
        ops.MultMatVec(A_nonlin, q),
        ops.MultVecScal(b_nonlin, -1)
    );

    return ops.DotProduct(r, r) / ops.DotProduct(b_nonlin, b_nonlin);
}

double Newton::CalcL2Error()
{
    double sum = 0.0;
    for (int i = 0; i < mesh.n_x; i++)
    {
        double diff = q[2 * i] - functions.u(mesh.meshX[i]);
        sum += diff * diff;
    }
    return std::sqrt(sum);
}

void Newton::PrintTimeLayerInfo(int layerIndex, int iterationCount)
{
    double error = CalcL2Error();
    std::streamsize oldPrecision = std::cout.precision();
    std::ios::fmtflags oldFlags = std::cout.flags();

    std::cout << std::fixed << std::setprecision(5)
        << "Time layer " << layerIndex << ": t = " << mesh.meshT[layerIndex]
        << ", countNodes = " << mesh.n_x;

    if (layerIndex > 0)
    {
        std::cout << ", ht = " << (mesh.meshT[layerIndex] - mesh.meshT[layerIndex - 1])
            << ", countIteration = " << iterationCount;
    }

    std::cout << std::scientific << std::setprecision(4)
        << ", ||u* - u||L2 = " << error << std::endl;

    std::cout.flags(oldFlags);
    std::cout.precision(oldPrecision);
}

```